

what is Express router explain easy for understanding

Here's a simple explanation of **Express Router**:

What is Express Router?

Think of Express Router as a **mini-application** inside your main Express app. It helps you organize your code by grouping related routes together.

A Real-World Analogy

Imagine you're organizing a big office:

- **Without Router:** All files (routes) piled in one huge cabinet → chaos
- **With Router:** Separate cabinets for "Sales," "HR," "Finance" → organized

Simple Example

Without Router (messy):

javascript

```
app.get('/users/profile', ...)
app.get('/users/settings', ...)
app.post('/users/create', ...)
app.get('/products/list', ...)
app.post('/products/buy', ...)
```

With Router (clean):

javascript

```
// users.js - User routes grouped together
const userRouter = express.Router()
userRouter.get('/profile', ...)
userRouter.get('/settings', ...)
userRouter.post('/create', ...)
```

```
// products.js - Product routes grouped together
const productRouter = express.Router()
productRouter.get('/list', ...)
productRouter.post('/buy', ...)

// Main app just "plugs in" the routers
app.use('/users', userRouter)
app.use('/products', productRouter)
```

Key Benefits

1. **Organization** - Group related routes (users, products, orders)
2. **Reusability** - Use same router in different parts of app
3. **Middleware** - Add specific middleware to certain route groups
4. **Clean Code** - Each file has one responsibility

When to Use It

- Your app has more than 10-15 routes
- Different features are logically separate
- You want to share route logic across multiple apps

Bottom line: Express Router is just a way to split your routes into smaller, manageable pieces instead of putting everything in one giant file.

using Express Router

Here's a **practical, step-by-step guide** to using Express Router in your projects:

1. Basic Setup

First, install Express:

```
bash

npm install express
```

2. Create Your Main App

app.js (main file):

javascript

```
const express = require('express')
const app = express()

// Import your routers
const userRouter = require('./routes/users')
const productRouter = require('./routes/products')

// Use the routers
app.use('/users', userRouter)    // ALL user routes start with /users
app.use('/products', productRouter) // ALL product routes start with /products

app.listen(3000, () => console.log('Server running'))
```

3. Create a Router File

routes/users.js :

javascript

```
const express = require('express')
const router = express.Router()

// Define routes (notice: no '/users' needed here)
router.get('/', (req, res) => {
  res.send('List of all users')
})

router.get('/:id', (req, res) => {
  res.send(`User with ID: ${req.params.id}`)
})

router.post('/', (req, res) => {
  res.send('Create a new user')
})

router.put('/:id', (req, res) => {
```

```
    res.send(`Update user ${req.params.id}`)
  })

  router.delete('/:id', (req, res) => {
    res.send(`Delete user ${req.params.id}`)
  })

  module.exports = router
```

4. Another Router Example

routes/products.js :

javascript

```
const express = require('express')
const router = express.Router()

router.get('/', (req, res) => {
  res.json([
    { id: 1, name: 'Laptop', price: 999 },
    { id: 2, name: 'Phone', price: 699 }
  ])
})

router.get('/search', (req, res) => {
  const { q } = req.query
  res.send(`Searching for: ${q}`)
})

module.exports = router
```

5. Using Router-Level Middleware

routes/admin.js :

javascript

```
const express = require('express')
const router = express.Router()
```

```

// Middleware specific to admin routes
router.use((req, res, next) => {
  console.log('Admin access at:', Date.now())
  next()
})

// Auth check middleware for specific routes
const checkAuth = (req, res, next) => {
  if (req.headers.authorization) {
    next()
  } else {
    res.status(401).send('Unauthorized')
  }
}

router.get('/dashboard', checkAuth, (req, res) => {
  res.send('Admin dashboard')
})

module.exports = router

```

6. Complete Example with Multiple Files

Project structure:

text

```

my-app/
├─ app.js
├─ routes/
│   ├─ users.js
│   ├─ products.js
│   └─ admin.js
└─ middleware/
    └─ auth.js

```

middleware/auth.js :

javascript

```

function isLoggedIn(req, res, next) {
  if (req.session.user) {

```

```
    next()
  } else {
    res.redirect('/login')
  }
}
```

```
module.exports = isLoggedIn
```

routes/users.js **(with middleware):**

javascript

```
const express = require('express')
const router = express.Router()
const isLoggedIn = require('../middleware/auth')

// Apply middleware to all routes in this router
router.use(isLoggedIn)

router.get('/profile', (req, res) => {
  res.send('User profile')
})

router.post('/update', (req, res) => {
  res.send('User updated')
})

module.exports = router
```

7. Nesting Routers (Advanced)

routes/api/v1/users.js :

javascript

```
const express = require('express')
const router = express.Router()

router.get('/', (req, res) => {
  res.json({ version: 'v1', users: [] })
})
```

```
module.exports = router
```

routes/api/v1.js :

```
javascript

const express = require('express')
const router = express.Router()
const userRouter = require('./v1/users')

router.use('/users', userRouter)

module.exports = router
```

app.js :

```
javascript

const v1Router = require('./routes/api/v1')
app.use('/api/v1', v1Router)

// Now your users are at: GET /api/v1/users/
```

URL Paths Summary

Router File	Router Path in app.js	Actual URL
router.get('/')	app.use('/users')	/users
router.get('/:id')	app.use('/users')	/users/123
router.get('/profile')	app.use('/users')	/users/profile
router.get('/')	app.use('/api/v1')	/api/v1

Key Points to Remember

- 1. Export your router with `module.exports = router`

2. **Don't repeat the base path** inside router files
3. **Use** `app.use()` to mount routers in main app
4. **Middleware can be added** at router level or route level
5. **Routers can be nested** for complex APIs

Run It

```
bash
```

```
node app.js
```

```
# Test endpoints:
```

```
# GET http://localhost:3000/users/5
```

```
# GET http://localhost:3000/products
```

```
# GET http://localhost:3000/products/search?q=Laptop
```

That's it! Express Router helps you keep your code organized, maintainable, and scalable as your app grows.

Restructuring Listings

It looks like you meant "**Restructuring Listings**" — likely referring to reorganizing your Express Router code for better structure, especially when dealing with listings (like product listings, user lists, or API endpoints).

I'll show you how to **restructure route listings** using Express Router, moving from messy to clean, organized code.

Problem: Messy Route Listings

`app.js` (BAD - everything piled together):

```
javascript
```

```
app.get('/products', (req, res) => { /* 50 lines */ })
app.get('/products/:id', (req, res) => { /* 50 lines */ })
app.post('/products', (req, res) => { /* 50 lines */ })
app.put('/products/:id', (req, res) => { /* 50 lines */ })
```

```

app.delete('/products/:id', (req, res) => { /* 50 lines */ })
app.get('/users', (req, res) => { /* 50 lines */ })
app.get('/users/:id', (req, res) => { /* 50 lines */ })
// ... 500+ lines of routes

```

Solution: Restructured Listings with Express Router

Step 1: Create a Clean Folder Structure

text

```

project/
├─ server.js (or app.js)
├─ routes/
│   ├─ index.js
│   ├─ productRoutes.js
│   ├─ userRoutes.js
│   └─ orderRoutes.js
├─ controllers/
│   ├─ productController.js
│   ├─ userController.js
│   └─ orderController.js
└─ middleware/
    └─ auth.js

```

Step 2: Create Route Files

routes/productRoutes.js :

javascript

```

const express = require('express')
const router = express.Router()
const productController = require('../controllers/productController')
const auth = require('../middleware/auth')

// GET /api/products - Get all products (listing)
router.get('/', productController.getAllProducts)

// GET /api/products/:id - Get single product

```

```
router.get('/:id', productController.getProductById)

// POST /api/products - Create product (protected)
router.post('/', auth, productController.createProduct)

// PUT /api/products/:id - Update product
router.put('/:id', auth, productController.updateProduct)

// DELETE /api/products/:id - Delete product
router.delete('/:id', auth, productController.deleteProduct)

module.exports = router
```

routes/userRoutes.js :

```
javascript

const express = require('express')
const router = express.Router()
const userController = require('../controllers/userController')

// GET /api/users - User Listing with pagination
router.get('/', userController.getAllUsers)

// GET /api/users/:id - Single user
router.get('/:id', userController.getUserById)

// POST /api/users - Create user
router.post('/', userController.createUser)

// PUT /api/users/:id - Update user
router.put('/:id', userController.updateUser)

// DELETE /api/users/:id - Delete user
router.delete('/:id', userController.deleteUser)

module.exports = router
```

Step 3: Create Controller Files (Business Logic)

controllers/productController.js :

javascript

```
// Mock database
let products = [
  { id: 1, name: 'Laptop', price: 999 },
  { id: 2, name: 'Phone', price: 699 },
  { id: 3, name: 'Tablet', price: 399 }
]

// GET all products with filtering, pagination, sorting
const getAllProducts = (req, res) => {
  let result = [...products]

  // Filter by price
  if (req.query.minPrice) {
    result = result.filter(p => p.price >= parseInt(req.query.minPrice))
  }

  // Pagination
  const page = parseInt(req.query.page) || 1
  const limit = parseInt(req.query.limit) || 10
  const startIndex = (page - 1) * limit
  const endIndex = page * limit

  const paginatedResult = result.slice(startIndex, endIndex)

  res.json({
    success: true,
    count: paginatedResult.length,
    total: result.length,
    page: page,
    totalPages: Math.ceil(result.length / limit),
    data: paginatedResult
  })
}

// GET single product
const getProductById = (req, res) => {
  const product = products.find(p => p.id === parseInt(req.params.id))

  if (!product) {
    return res.status(404).json({ success: false, message: 'Product not found' })
  }
}
```

```
res.json({ success: true, data: product })
}

// CREATE product
const createProduct = (req, res) => {
  const { name, price } = req.body

  if (!name || !price) {
    return res.status(400).json({ success: false, message: 'Name and price required' })
  }

  const newProduct = {
    id: products.length + 1,
    name,
    price
  }

  products.push(newProduct)
  res.status(201).json({ success: true, data: newProduct })
}

// UPDATE product
const updateProduct = (req, res) => {
  const id = parseInt(req.params.id)
  const { name, price } = req.body
  const productIndex = products.findIndex(p => p.id === id)

  if (productIndex === -1) {
    return res.status(404).json({ success: false, message: 'Product not found' })
  }

  products[productIndex] = { ...products[productIndex], name, price }
  res.json({ success: true, data: products[productIndex] })
}

// DELETE product
const deleteProduct = (req, res) => {
  const id = parseInt(req.params.id)
  const productIndex = products.findIndex(p => p.id === id)

  if (productIndex === -1) {
    return res.status(404).json({ success: false, message: 'Product not found' })
  }
}
```

```
products.splice(productIndex, 1)
res.json({ success: true, message: 'Product deleted' })
}

module.exports = {
  getAllProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
}
```

Step 4: Combine All Routes in Index

routes/index.js :

javascript

```
const express = require('express')
const router = express.Router()

// Import all route modules
const productRoutes = require('./productRoutes')
const userRoutes = require('./userRoutes')
const orderRoutes = require('./orderRoutes')

// Mount routes with their base paths
router.use('/products', productRoutes) // ALL product routes
router.use('/users', userRoutes)       // ALL user routes
router.use('/orders', orderRoutes)     // ALL order routes

// Health check endpoint
router.get('/health', (req, res) => {
  res.json({ status: 'OK', timestamp: new Date() })
})

module.exports = router
```

Step 5: Main App File (Clean!)

server.js :

javascript

```
const express = require('express')
const app = express()

// Middleware
app.use(express.json()) // Parse JSON bodies
app.use(express.urlencoded({ extended: true }))

// Import all routes at once
const apiRoutes = require('./routes')

// Use the routes - all API endpoints start with /api
app.use('/api', apiRoutes)

// 404 handler for unmatched routes
app.use((req, res) => {
  res.status(404).json({ success: false, message: 'Route not found' })
})

// Error handler
app.use((err, req, res, next) => {
  console.error(err)
  res.status(500).json({ success: false, message: 'Server error' })
})

app.listen(3000, () => console.log('Server running on port 3000'))
```

Benefits of This Restructured Listing

Before (Monolithic):

javascript

```
// Everything in one file - hard to find and maintain
app.get('/api/products', ...)
app.get('/api/products/:id', ...)
app.post('/api/products', ...)
app.get('/api/users', ...)
// 500 more lines...
```

After (Modular):

```
javascript
```

```
// Clean, organized, easy to navigate
app.use('/api/products', productRoutes) // products logic in productRoutes.js
app.use('/api/users', userRoutes)      // users logic in userRoutes.js
```

Testing Your Restructured API

```
bash
```

```
# Test product listings with pagination
GET http://localhost:3000/api/products?page=1&limit=5

# Test with filtering
GET http://localhost:3000/api/products?minPrice=500

# Get single product
GET http://localhost:3000/api/products/1

# Create product
POST http://localhost:3000/api/products
Content-Type: application/json
{
  "name": "Keyboard",
  "price": 49
}
```

Summary of Best Practices

1. **Separate concerns:** Routes → Controllers → Models
2. **Use** router for each resource (products, users, orders)
3. **Mount at** index.js then mount once in main app
4. **Keep controllers** for business logic (not in route files)
5. **Use consistent naming:** productRoutes.js , userController.js
6. **Add pagination** for listing endpoints
7. **Return consistent JSON structure** (success, data, message)

This structure makes your code:

- **Easy to navigate** (find products logic in products folder)
- **Easy to test** (test controllers separately)
- **Easy to scale** (add new features without breaking existing)
- **Team-friendly** (multiple devs work on different routes)

Restructuring Reviews

It looks like you're asking about **restructuring review functionality** in an Express.js application — similar to the "listings" example, but now for a **reviews** feature (like product reviews, user reviews, or rating systems).

I'll show you how to restructure **review routes, controllers, and logic** using Express Router for a clean, maintainable codebase.

Complete Review System Restructuring

Project Structure for Reviews

text

```
project/
├─ server.js
├─ routes/
│   ├─ index.js
│   ├─ reviewRoutes.js
│   ├─ productRoutes.js
│   └─ userRoutes.js
├─ controllers/
│   ├─ reviewController.js
│   ├─ productController.js
│   └─ userController.js
├─ middleware/
│   ├─ auth.js
│   └─ validation.js
└─ models/
    └─ Review.js (if using database)
```

Step 1: Review Routes (Clean & Organized)

routes/reviewRoutes.js :

javascript

```
const express = require('express')
const router = express.Router()
const reviewController = require('../controllers/reviewController')
const { protect, restrictTo } = require('../middleware/auth')
const { validateReview } = require('../middleware/validation')

// Public routes (anyone can view reviews)
router.get('/', reviewController.getAllReviews)
router.get('/:id', reviewController.getReviewById)

// Product-specific reviews
router.get('/product/:productId', reviewController.getReviewsByProduct)
router.get('/user/:userId', reviewController.getReviewsByUser)

// Protected routes (only logged-in users)
router.use(protect) // All routes below require login

// Create review (must be logged in)
router.post('/', validateReview, reviewController.createReview)

// Update/Delete own review
router.put('/:id', validateReview, reviewController.updateReview)
router.delete('/:id', reviewController.deleteReview)

// Admin only
router.delete('/admin/:id', restrictTo('admin'), reviewController.adminDeleteReview)
router.put('/admin/:id/approve', restrictTo('admin'), reviewController.approveReview)

module.exports = router
```

Step 2: Review Controller (Business Logic)

controllers/reviewController.js :

javascript

```
// Mock database (replace with actual DB like MongoDB/PostgreSQL)
```

```
let reviews = [  
  {  
    id: 1,  
    productId: 101,  
    userId: 1,  
    userName: 'John Doe',  
    rating: 5,  
    title: 'Excellent product!',  
    comment: 'Really love this laptop, works perfectly.',  
    createdAt: '2024-01-15',  
    status: 'approved'  
  },  
  {  
    id: 2,  
    productId: 101,  
    userId: 2,  
    userName: 'Jane Smith',  
    rating: 4,  
    title: 'Good but expensive',  
    comment: 'Quality is great but price is high.',  
    createdAt: '2024-01-20',  
    status: 'approved'  
  },  
  {  
    id: 3,  
    productId: 102,  
    userId: 3,  
    userName: 'Bob Wilson',  
    rating: 2,  
    title: 'Disappointed',  
    comment: 'Phone battery drains too fast.',  
    createdAt: '2024-01-25',  
    status: 'pending'  
  }  
]
```

```
// GET all reviews (with filtering, pagination, sorting)
```

```
const getAllReviews = (req, res) => {  
  let result = [...reviews]  
  
  // Filter by status (if admin)  
  if (req.query.status && req.user?.role === 'admin') {
```

```
    result = result.filter(r => r.status === req.query.status)
  } else {
    // Non-admins only see approved reviews
    result = result.filter(r => r.status === 'approved')
  }

  // Filter by rating
  if (req.query.minRating) {
    result = result.filter(r => r.rating >= parseInt(req.query.minRating))
  }

  // Sorting
  const sortBy = req.query.sortBy || 'createdAt'
  const order = req.query.order === 'asc' ? 1 : -1
  result.sort((a, b) => (a[sortBy] > b[sortBy] ? order : -order))

  // Pagination
  const page = parseInt(req.query.page) || 1
  const limit = parseInt(req.query.limit) || 10
  const startIndex = (page - 1) * limit
  const endIndex = page * limit

  const paginatedResult = result.slice(startIndex, endIndex)

  // Calculate average rating
  const avgRating = result.length > 0
    ? result.reduce((sum, r) => sum + r.rating, 0) / result.length
    : 0

  res.json({
    success: true,
    stats: {
      totalReviews: result.length,
      averageRating: avgRating.toFixed(1),
      currentPage: page,
      totalPages: Math.ceil(result.length / limit)
    },
    data: paginatedResult
  })
}

// GET single review by ID
const getReviewById = (req, res) => {
  const review = reviews.find(r => r.id === parseInt(req.params.id))
```

```
if (!review) {
  return res.status(404).json({
    success: false,
    message: 'Review not found'
  })
}

// Hide pending reviews from non-admins
if (review.status !== 'approved' && req.user?.role !== 'admin') {
  return res.status(403).json({
    success: false,
    message: 'Review not available'
  })
}

res.json({ success: true, data: review })
}

// GET reviews by product
const getReviewsByProduct = (req, res) => {
  const productId = parseInt(req.params.productId)

  let productReviews = reviews.filter(r => r.productId === productId)

  // Filter by status
  if (req.user?.role !== 'admin') {
    productReviews = productReviews.filter(r => r.status === 'approved')
  }

  const avgRating = productReviews.length > 0
    ? productReviews.reduce((sum, r) => sum + r.rating, 0) / productReviews.length
    : 0

  res.json({
    success: true,
    productId: productId,
    stats: {
      total: productReviews.length,
      averageRating: avgRating.toFixed(1)
    },
    data: productReviews
  })
}
```

```
// GET reviews by user
const getReviewsByUser = (req, res) => {
  const userId = parseInt(req.params.userId)

  // Users can only see their own pending reviews
  const isOwnProfile = req.user?.id === userId || req.user?.role === 'admin'

  let userReviews = reviews.filter(r => r.userId === userId)

  if (!isOwnProfile) {
    userReviews = userReviews.filter(r => r.status === 'approved')
  }

  res.json({
    success: true,
    data: userReviews
  })
}

// CREATE new review
const createReview = (req, res) => {
  const { productId, rating, title, comment } = req.body

  // Check if user already reviewed this product
  const existingReview = reviews.find(r =>
    r.userId === req.user.id && r.productId === productId
  )

  if (existingReview) {
    return res.status(400).json({
      success: false,
      message: 'You have already reviewed this product'
    })
  }

  const newReview = {
    id: reviews.length + 1,
    productId: parseInt(productId),
    userId: req.user.id,
    userName: req.user.name,
    rating: parseInt(rating),
    title,
    comment,
  }
```

```
    createdAt: new Date().toISOString().split('T')[0],
    status: 'pending' // Requires admin approval
  }

  reviews.push(newReview)

  res.status(201).json({
    success: true,
    message: 'Review submitted for approval',
    data: newReview
  })
}

// UPDATE own review
const updateReview = (req, res) => {
  const id = parseInt(req.params.id)
  const { rating, title, comment } = req.body

  const reviewIndex = reviews.findIndex(r => r.id === id)

  if (reviewIndex === -1) {
    return res.status(404).json({ success: false, message: 'Review not found' })
  }

  const review = reviews[reviewIndex]

  // Check ownership
  if (review.userId !== req.user.id && req.user.role !== 'admin') {
    return res.status(403).json({
      success: false,
      message: 'You can only update your own reviews'
    })
  }

  // Update review
  reviews[reviewIndex] = {
    ...review,
    rating: parseInt(rating) || review.rating,
    title: title || review.title,
    comment: comment || review.comment,
    status: 'pending', // Reset to pending for re-approval
    updatedAt: new Date().toISOString().split('T')[0]
  }
}
```

```
    res.json({
      success: true,
      message: 'Review updated and pending approval',
      data: reviews[reviewIndex]
    })
  }

// DELETE own review
const deleteReview = (req, res) => {
  const id = parseInt(req.params.id)
  const reviewIndex = reviews.findIndex(r => r.id === id)

  if (reviewIndex === -1) {
    return res.status(404).json({ success: false, message: 'Review not found' })
  }

  const review = reviews[reviewIndex]

  // Check ownership
  if (review.userId !== req.user.id && req.user.role !== 'admin') {
    return res.status(403).json({
      success: false,
      message: 'You can only delete your own reviews'
    })
  }

  reviews.splice(reviewIndex, 1)

  res.json({
    success: true,
    message: 'Review deleted successfully'
  })
}

// ADMIN: Delete any review
const adminDeleteReview = (req, res) => {
  const id = parseInt(req.params.id)
  const reviewIndex = reviews.findIndex(r => r.id === id)

  if (reviewIndex === -1) {
    return res.status(404).json({ success: false, message: 'Review not found' })
  }

  reviews.splice(reviewIndex, 1)
```

```
    res.json({
      success: true,
      message: 'Review deleted by admin'
    })
  }

  // ADMIN: Approve review
  const approveReview = (req, res) => {
    const id = parseInt(req.params.id)
    const reviewIndex = reviews.findIndex(r => r.id === id)

    if (reviewIndex === -1) {
      return res.status(404).json({ success: false, message: 'Review not found' })
    }

    reviews[reviewIndex].status = 'approved'

    res.json({
      success: true,
      message: 'Review approved',
      data: reviews[reviewIndex]
    })
  }

  module.exports = {
    getAllReviews,
    getReviewById,
    getReviewsByProduct,
    getReviewsByUser,
    createReview,
    updateReview,
    deleteReview,
    adminDeleteReview,
    approveReview
  }
```

Step 3: Authentication Middleware

middleware/auth.js :

javascript

```
// Mock authentication (replace with JWT/session)
const protect = async (req, res, next) => {
  // Get token from header
  const token = req.headers.authorization?.split(' ')[1]

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Not authorized, no token'
    })
  }

  try {
    // Verify token and attach user to request
    // For demo purposes, using mock user
    req.user = {
      id: 1,
      name: 'John Doe',
      email: 'john@example.com',
      role: 'user' // or 'admin'
    }
    next()
  } catch (error) {
    res.status(401).json({ success: false, message: 'Invalid token' })
  }
}

const restrictTo = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: 'You do not have permission'
      })
    }
    next()
  }
}

module.exports = { protect, restrictTo }
```

Step 4: Validation Middleware

middleware/validation.js :

javascript

```
const validateReview = (req, res, next) => {
  const { productId, rating, title, comment } = req.body

  const errors = []

  if (!productId) {
    errors.push('Product ID is required')
  }

  if (!rating || rating < 1 || rating > 5) {
    errors.push('Rating must be between 1 and 5')
  }

  if (!title || title.length < 3 || title.length > 100) {
    errors.push('Title must be between 3 and 100 characters')
  }

  if (!comment || comment.length < 10 || comment.length > 500) {
    errors.push('Comment must be between 10 and 500 characters')
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      errors
    })
  }

  next()
}

module.exports = { validateReview }
```

Step 5: Integrate Reviews with Products

routes/productRoutes.js (nested reviews):

javascript

```
const express = require('express')
const router = express.Router()
const productController = require('../controllers/productController')
const reviewController = require('../controllers/reviewController')
const { protect } = require('../middleware/auth')

// Product routes
router.get('/', productController.getAllProducts)
router.get('/:id', productController.getProductById)

// Nested review routes (GET /products/:id/reviews)
router.get('/:productId/reviews', reviewController.getReviewsByProduct)
router.post('/:productId/reviews', protect, reviewController.createReview)

module.exports = router
```

Step 6: Main App File

server.js :

javascript

```
const express = require('express')
const app = express()

app.use(express.json())

// Import routes
const apiRoutes = require('./routes')
const reviewRoutes = require('./routes/reviewRoutes')
const productRoutes = require('./routes/productRoutes')

// Mount routes
app.use('/api', apiRoutes)
app.use('/api/reviews', reviewRoutes)
app.use('/api/products', productRoutes)

// Start server
app.listen(3000, () => {
  console.log('Server running on port 3000')
})
```

Testing Your Review System

bash

Get all approved reviews (public)

GET http://localhost:3000/api/reviews

Get reviews for specific product

GET http://localhost:3000/api/reviews/product/101

Get reviews with filters

GET http://localhost:3000/api/reviews?minRating=4&sortBy=rating

Create a review (needs auth token)

POST http://localhost:3000/api/reviews

Authorization: Bearer YOUR_TOKEN

Content-Type: application/json

```
{
  "productId": 101,
  "rating": 5,
  "title": "Amazing product!",
  "comment": "Best purchase I've ever made. Highly recommended!"
}
```

Update your review

PUT http://localhost:3000/api/reviews/1

Authorization: Bearer YOUR_TOKEN

```
{
  "rating": 4,
  "comment": "Still good but found some issues"
}
```

Delete your review

DELETE http://localhost:3000/api/reviews/1

Authorization: Bearer YOUR_TOKEN

Admin: Approve review

PUT http://localhost:3000/api/reviews/admin/3/approve

Authorization: Bearer ADMIN_TOKEN

Key Features of This Restructuring

1. **Separation of concerns** - Routes, controllers, middleware all separate
2. **CRUD operations** - Full create, read, update, delete for reviews
3. **Authorization** - Users can only edit their own reviews
4. **Approval workflow** - Reviews need admin approval
5. **Filtering & pagination** - Efficient data fetching
6. **Nested routes** - `/products/:id/reviews` for direct access
7. **Validation** - Input sanitization and error handling
8. **Statistics** - Average ratings, total counts

This structure makes your review system **scalable, maintainable, and easy to test!**

web Cookies

Here's an **easy-to-understand explanation** of web cookies, including how they work with Express.js.

What Are Web Cookies?

Simple definition: Cookies are small text files stored in your browser that websites use to **remember information** about you.

Real-World Analogy

Imagine you visit a coffee shop:

- **Without cookies:** The barista asks your name every single time you order ☕
- **With cookies:** The barista gives you a loyalty card. Each time you visit, they scan it and instantly know your name, favorite drink, and rewards status

The cookie is that loyalty card — it helps websites recognize you between visits.

What Cookies Do

Feature	What It Does	Example
Remember you	Keeps you logged in	Stay logged into Facebook for weeks
Remember preferences	Saves your settings	Dark mode, language preference
Track behavior	What you click and buy	Amazon recommendations
Shopping cart	Items stay in cart	Add item to cart, come back tomorrow

How Cookies Work (Simple Flow)

text

- 1. You visit website →
- 2. Server sends cookie to your browser →
- 3. Browser stores cookie →
- 4. You visit again →
- 5. Browser sends cookie back to server →
- 6. Server recognizes you

Types of Cookies

javascript

- // 1. Session Cookies (delete when browser closes)
- // 2. Persistent Cookies (stay for days/weeks)
- // 3. Secure Cookies (only over HTTPS)
- // 4. HttpOnly Cookies (can't be read by JavaScript - more secure)

Express.js Cookie Examples

Step 1: Install Cookie Parser

bash

```
npm install cookie-parser
```

Step 2: Basic Cookie Setup

javascript

```
const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

app.use(cookieParser()) // Enable cookie parsing

// SET a cookie
app.get('/set-cookie', (req, res) => {
  res.cookie('username', 'JohnDoe', {
    maxAge: 900000, // Expires in 15 minutes (milliseconds)
    httpOnly: true, // Can't be accessed by JavaScript
    secure: false, // Set true for HTTPS only
  })

  res.send('Cookie has been set!')
})

// READ a cookie
app.get('/get-cookie', (req, res) => {
  const username = req.cookies.username
  res.send(`Cookie value is: ${username}`)
})

// DELETE a cookie
app.get('/delete-cookie', (req, res) => {
  res.clearCookie('username')
  res.send('Cookie deleted!')
})

app.listen(3000)
```

Real Example: Login System with Cookies

app.js :

javascript

```
const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: true }))
app.use(cookieParser())

// Login page (HTML form)
app.get('/login', (req, res) => {
  res.send(`
    <html>
      <body>
        <h2>Login</h2>
        <form method="POST" action="/login">
          <input type="text" name="username" placeholder="Username" required />
          <input type="password" name="password" placeholder="Password" required />
          <label>
            <input type="checkbox" name="remember" value="yes" />
            Remember me for 7 days
          </label>
          <button type="submit">Login</button>
        </form>
      </body>
    </html>
  `)
})

// Handle Login
app.post('/login', (req, res) => {
  const { username, password, remember } = req.body

  // Simple validation (use real auth in production)
  if (username === 'admin' && password === '123') {

    // Set cookie options based on "remember me"
    const cookieOptions = {}

    if (remember === 'yes') {
      // 7 days persistent cookie
      cookieOptions.maxAge = 7 * 24 * 60 * 60 * 1000 // 7 days in milliseconds
      cookieOptions.httpOnly = true
    }
  }
})
```

```
    } else {
      // Session cookie (deletes when browser closes)
      cookieOptions.httpOnly = true
    }

    // Set authentication cookie
    res.cookie('auth', username, cookieOptions)

    res.send(`
      <h2>Welcome ${username}!</h2>
      <p>You are now logged in.</p>
      <a href="/dashboard">Go to Dashboard</a>
      <br/><br/>
      <a href="/logout">Logout</a>
    `)
  } else {
    res.send('<h2>Invalid credentials! <a href="/login">Try again</a></h2>')
  }
})

// Protected dashboard (checks cookie)
app.get('/dashboard', (req, res) => {
  const username = req.cookies.auth

  if (!username) {
    return res.send(`
      <h2>You are not logged in!</h2>
      <a href="/login">Login here</a>
    `)
  }

  res.send(`
    <h2>Dashboard</h2>
    <p>Welcome back, ${username}!</p>
    <p>This is your private dashboard.</p>
    <a href="/logout">Logout</a>
  `)
})

// Logout - clear cookie
app.get('/logout', (req, res) => {
  res.clearCookie('auth')
  res.send(`
    <h2>You have been logged out</h2>
  `)
```

```

    <a href="/login">Login again</a>
  `)
})

```

```
app.listen(3000, () => console.log('Server running on http://localhost:3000'))
```

Enhanced Example: Shopping Cart with Cookies

cart-app.js :

javascript

```

const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

app.use(cookieParser())
app.use(express.urlencoded({ extended: true }))

// Products data
const products = [
  { id: 1, name: 'Laptop', price: 999 },
  { id: 2, name: 'Mouse', price: 29 },
  { id: 3, name: 'Keyboard', price: 79 }
]

// Helper to get cart from cookie
function getCart(req) {
  const cart = req.cookies.cart
  return cart ? JSON.parse(cart) : []
}

// Helper to save cart to cookie
function saveCart(res, cart) {
  res.cookie('cart', JSON.stringify(cart), {
    maxAge: 30 * 24 * 60 * 60 * 1000, // 30 days
    httpOnly: true
  })
}

// Product Listing page
app.get('/products', (req, res) => {
  const cart = getCart(req)

```

```

const cartCount = cart.reduce((sum, item) => sum + item.quantity, 0)

let html = `
  <h2>Products</h2>
  <p>🛒 Cart has ${cartCount} item(s) | <a href="/cart">View Cart</a></p>
  <ul>
    `

products.forEach(product => {
  html += `
    <li>
      <strong>${product.name}</strong> - ${product.price}
      <form method="POST" action="/add-to-cart" style="display:inline">
        <input type="hidden" name="productId" value="${product.id}" />
        <input type="number" name="quantity" value="1" min="1" style="width:50px" />
        <button type="submit">Add to Cart</button>
      </form>
    </li>
    `
})

html += `</ul><a href="/checkout">Checkout</a>`
res.send(html)
})

// Add to cart
app.post('/add-to-cart', (req, res) => {
  const { productId, quantity } = req.body
  const cart = getCart(req)

  const product = products.find(p => p.id === parseInt(productId))

  if (!product) {
    return res.send('Product not found')
  }

  const existingItem = cart.find(item => item.id === parseInt(productId))

  if (existingItem) {
    existingItem.quantity += parseInt(quantity)
  } else {
    cart.push({
      id: product.id,
      name: product.name,

```

```

        price: product.price,
        quantity: parseInt(quantity)
      })
    }

    saveCart(res, cart)
    res.redirect('/products')
  })

  // View cart
  app.get('/cart', (req, res) => {
    const cart = getCart(req)

    if (cart.length === 0) {
      return res.send(`
        <h2>Your cart is empty</h2>
        <a href="/products">Continue Shopping</a>
      `)
    }

    let total = 0
    let html = `<h2>Your Shopping Cart</h2><table border="1" cellpadding="10">`
    html += `<tr><th>Product</th><th>Price</th><th>Quantity</th><th>Subtotal</th></tr>`

    cart.forEach(item => {
      const subtotal = item.price * item.quantity
      total += subtotal
      html += `
        <tr>
          <td>${item.name}</td>
          <td>${item.price}</td>
          <td>${item.quantity}</td>
          <td>${subtotal}</td>
        </tr>
      `
    })

    html += `<tr><td colspan="3"><strong>Total</strong></td><td><strong>${total}</strong>
</td></tr>`

    html += `</table><br/><a href="/products">Continue Shopping</a> | <a href="/clear-cart">Clear Cart</a>`

    res.send(html)
  })

```

```

// Clear cart
app.get('/clear-cart', (req, res) => {
  res.clearCookie('cart')
  res.redirect('/products')
})

// Checkout
app.get('/checkout', (req, res) => {
  const cart = getCart(req)

  if (cart.length === 0) {
    return res.send('<h2>Cart is empty! <a href="/products">Shop now</a></h2>')
  }

  res.send(`
    <h2>Checkout</h2>
    <p>Total items: ${cart.reduce((s, i) => s + i.quantity, 0)}</p>
    <form method="POST" action="/checkout">
      <input type="text" placeholder="Name" required /><br/><br/>
      <input type="text" placeholder="Address" required /><br/><br/>
      <button type="submit">Place Order</button>
    </form>
  `)
})

app.post('/checkout', (req, res) => {
  res.clearCookie('cart')
  res.send('<h2>Order placed successfully!</h2><a href="/products">Shop More</a>')
})

app.listen(3000)

```

Cookie Options Explained

javascript

```

res.cookie('name', 'value', {
  maxAge: 900000,      // Milliseconds until expiry (15 min)
  expires: new Date('2024-12-31'), // Specific expiry date
  httpOnly: true,      // Can't be read by JavaScript (more secure)
  secure: true,        // Only send over HTTPS
  sameSite: 'lax',     // CSRF protection ('strict', 'lax', 'none')
})

```

```
domain: '.example.com', // Which domain can read cookie
path: '/admin'           // Only send to specific paths
})
```

View Cookies in Browser

Chrome/Edge:

1. Press **F12** to open DevTools
2. Go to **Application** tab
3. Click **Cookies** under Storage
4. See all cookies for current site

Firefox:

1. Press **F12**
2. Go to **Storage** tab
3. Click **Cookies**

Security Best Practices

javascript

```
// ✅ GOOD: HttpOnly + Secure + SameSite
res.cookie('session', token, {
  httpOnly: true,    // Prevent XSS attacks
  secure: true,       // Only HTTPS (production)
  sameSite: 'strict', // Prevent CSRF attacks
  maxAge: 24 * 60 * 60 * 1000 // 1 day
})

// ❌ BAD: No security flags
res.cookie('session', token) // Vulnerable to attacks

// ❌ BAD: Storing sensitive data in cookies
res.cookie('password', '123456') // Never do this!
```

```
//  GOOD: Store only session ID/token
res.cookie('sessionId', 'abc123xyz') // Reference to server-stored data
```

Common Cookie Use Cases

Use Case	Cookie Type	Duration
Login session	HttpOnly	Session or days
Remember Me	Persistent	7-30 days
Shopping cart	Persistent	30 days
Language preference	Persistent	1 year
Analytics tracking	Persistent	2 years
GDPR consent	Persistent	1 year

Summary

Cookies = Browser memory that websites can use

- **Store** small pieces of data (max 4KB per cookie)
- **Remember** users between visits
- **Track** behavior and preferences
- **Secure** with httpOnly, secure, sameSite flags
- **Express** makes it easy with `cookie-parser`

Remember: Never store sensitive data like passwords or credit cards in cookies — use server-side sessions instead!

sending cookies

Here's a **complete guide to sending cookies** in Express.js, from basic to advanced.

What Does "Sending Cookies" Mean?

Sending a cookie = Your server telling the browser "Please store this piece of data" by including it in the HTTP response headers.

How Cookies Are Sent (The Technical View)

When your server sends a response, it includes a `Set-Cookie` header:

```
http
```

```
HTTP/1.1 200 OK
```

```
Set-Cookie: username=John; Path=/; HttpOnly
```

```
Set-Cookie: theme=dark; Max-Age=86400
```

```
Content-Type: text/html
```

```
<html>...</html>
```

The browser reads this header and stores the cookie.

Basic Cookie Sending in Express

Setup First

```
bash
```

```
npm install express cookie-parser
```

```
javascript
```

```
const express = require('express')
```

```
const cookieParser = require('cookie-parser')
```

```
const app = express()
```

```
app.use(cookieParser()) // Required to READ cookies (not to SEND them)
```

```
// SENDING cookies doesn't require cookie-parser!
```

```
// But you need it to read incoming cookies
```

Method 1: Simple Cookie

```
javascript

// SEND a basic cookie
app.get('/send-simple-cookie', (req, res) => {
  res.cookie('user', 'Alice')
  res.send('Cookie sent! Check browser dev tools > Application > Cookies')
})
```

Method 2: Cookie with Options

```
javascript

app.get('/send-advanced-cookie', (req, res) => {
  res.cookie('preferences', JSON.stringify({ theme: 'dark', fontSize: 16 })), {
    maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days
    httpOnly: true,                  // Can't be read by JavaScript
    secure: false,                   // Set true for HTTPS only
    sameSite: 'lax',                 // CSRF protection
    path: '/',                       // Available everywhere
  })

  res.send('Cookie with options sent!')
})
```

Complete Examples: Sending Different Types of Cookies

Example 1: Simple Key-Value Cookie

```
javascript

const express = require('express')
const app = require('cookie-parser')

app.use(cookieParser())

// Send multiple cookies at once
app.get('/send-multiple', (req, res) => {
  res.cookie('username', 'JohnDoe')
```

```
res.cookie('theme', 'dark-mode')
res.cookie('language', 'en-US')

res.send('3 cookies sent!')
})

app.listen(3000)
```

Example 2: Cookie with Expiration

javascript

```
app.get('/session-cookie', (req, res) => {
  // Session cookie - deletes when browser closes
  res.cookie('sessionId', 'xyz789', {
    httpOnly: true
  })
  res.send('Session cookie sent (deletes when you close browser)')
})

app.get('/persistent-cookie', (req, res) => {
  // Persistent cookie - stays for 30 days
  res.cookie('rememberMe', 'true', {
    maxAge: 30 * 24 * 60 * 60 * 1000, // 30 days in milliseconds
    httpOnly: true
  })
  res.send('Persistent cookie sent (lasts 30 days)')
})

app.get('/expires-cookie', (req, res) => {
  // Cookie that expires on specific date
  const expiryDate = new Date('2025-12-31')

  res.cookie('limited', 'value', {
    expires: expiryDate
  })

  res.send(`Cookie expires on ${expiryDate.toDateString()}`)
})
```

Example 3: Signed Cookies (Secure)

javascript

```
// Setup with secret for signed cookies
app.use(cookieParser('my_super_secret_key_123'))

// Send signed cookie
app.get('/send-signed-cookie', (req, res) => {
  res.cookie('userId', '12345', {
    signed: true, // This signs the cookie to prevent tampering
    httpOnly: true
  })

  res.send('Signed cookie sent! Cannot be tampered with by client')
})

// Read signed cookie
app.get('/read-signed', (req, res) => {
  const signedCookie = req.signedCookies.userId

  if (signedCookie) {
    res.send(`Signed cookie value: ${signedCookie}`)
  } else {
    res.send('No signed cookie found or it was tampered with')
  }
})
```

Real-World Example: Complete User Session with Cookies

server.js :

javascript

```
const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: true }))
app.use(cookieParser('your-secret-key')) // Secret for signed cookies

// Login page (HTML form)
app.get('/', (req, res) => {
  const userId = req.signedCookies.userId
```

```

if (userId) {
  return res.send(`
    <h2>Welcome back, User ${userId}!</h2>
    <p>You are already logged in.</p>
    <a href="/logout">Logout</a>
  `)
}

res.send(`
  <html>
    <body>
      <h2>Login</h2>
      <form method="POST" action="/login">
        <input type="text" name="username" placeholder="Username" required />
        <input type="password" name="password" placeholder="Password" required />

        <h3>Cookie Preferences:</h3>
        <label>
          <input type="radio" name="cookieDuration" value="session" checked />
          Session cookie (expires when browser closes)
        </label><br/>
        <label>
          <input type="radio" name="cookieDuration" value="day" />
          Remember me for 1 day
        </label><br/>
        <label>
          <input type="radio" name="cookieDuration" value="week" />
          Remember me for 1 week
        </label><br/>
        <label>
          <input type="radio" name="cookieDuration" value="month" />
          Remember me for 1 month
        </label><br/><br/>

        <button type="submit">Login</button>
      </form>
    </body>
  </html>
  `)
})

```

```

// Process login and send appropriate cookie
app.post('/login', (req, res) => {

```

```
const { username, password, cookieDuration } = req.body

// Simple authentication (use real auth in production)
if (username === 'demo' && password === '123') {

  // Generate a simple user ID (in real app, use session ID from database)
  const userId = Math.floor(Math.random() * 10000)

  // Set cookie options based on user choice
  let cookieOptions = {
    signed: true,      // Sign the cookie
    httpOnly: true,    // Prevent XSS
    sameSite: 'lax',   // CSRF protection
    path: '/'
  }

  // Set expiration based on user selection
  switch(cookieDuration) {
    case 'day':
      cookieOptions.maxAge = 24 * 60 * 60 * 1000 // 1 day
      break
    case 'week':
      cookieOptions.maxAge = 7 * 24 * 60 * 60 * 1000 // 7 days
      break
    case 'month':
      cookieOptions.maxAge = 30 * 24 * 60 * 60 * 1000 // 30 days
      break
    default:
      // Session cookie (no maxAge)
      break
  }

  // SEND the cookie
  res.cookie('userId', userId, cookieOptions)

  // Also send a preference cookie
  res.cookie('preferences', JSON.stringify({
    theme: 'light',
    language: 'en',
    lastLogin: new Date().toISOString()
  }), {
    maxAge: 30 * 24 * 60 * 60 * 1000,
    httpOnly: false // Can be read by JavaScript
  })
}
```

```

// Send success response
res.send(`
  <h2>Login Successful!</h2>
  <p>Welcome ${username}!</p>
  <p>User ID: ${userId}</p>
  <p>Cookie duration: ${cookieDuration || 'session'}</p>
  <h3>Cookies sent:</h3>
  <ul>
    <li><strong>userId</strong> (signed, httpOnly)</li>
    <li><strong>preferences</strong> (JSON data)</li>
  </ul>
  <a href="/dashboard">Go to Dashboard</a><br/><br/>
  <a href="/view-cookies">View All Cookies</a><br/><br/>
  <a href="/logout">Logout</a>
`)
} else {
  res.send('<h2>Invalid credentials!</h2><a href="/">Try again</a>')
}
})

// Dashboard - reads cookies
app.get('/dashboard', (req, res) => {
  const userId = req.signedCookies.userId

  if (!userId) {
    return res.redirect('/')
  }

  let preferences = {}
  try {
    preferences = JSON.parse(req.cookies.preferences || '{}')
  } catch(e) {}

  res.send(`
    <h2>User Dashboard</h2>
    <p>User ID: ${userId}</p>
    <p>Preferences: ${JSON.stringify(preferences)}</p>
    <a href="/">Home</a> | <a href="/logout">Logout</a>
  `)
})

// View all cookies (for debugging)
app.get('/view-cookies', (req, res) => {

```

```
const cookies = req.cookies
const signedCookies = req.signedCookies

let html = '<h2>All Cookies in Browser</h2>'

html += '<h3>Regular Cookies:</h3><ul>'
for (let [name, value] of Object.entries(cookies)) {
  html += `<li><strong>${name}</strong>: ${value}</li>`
}
html += '</ul>'

html += '<h3>Signed Cookies:</h3><ul>'
for (let [name, value] of Object.entries(signedCookies)) {
  html += `<li><strong>${name}</strong>: ${value}</li>`
}
html += '</ul>'

html += '<br/><a href="/">Back to Home</a>'

res.send(html)
})

// Logout - clear cookies
app.get('/logout', (req, res) => {
  // Clear specific cookies
  res.clearCookie('userId')
  res.clearCookie('preferences')

  // Clear all cookies (loop through all)
  for (let cookieName in req.cookies) {
    res.clearCookie(cookieName)
  }

  res.send(`
    <h2>Logged out successfully!</h2>
    <p>All cookies have been cleared.</p>
    <a href="/">Login again</a>
  `)
})

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000')
```

```
console.log('Try the login with username: demo, password: 123')
})
```

Sending Cookies in Different Scenarios

1. Sending Cookie with JSON Response

javascript

```
app.post('/api/login', (req, res) => {
  const { email, password } = req.body

  if (email === 'user@example.com' && password === 'pass123') {
    // Send cookie AND JSON response
    res.cookie('authToken', 'abc123xyz', {
      httpOnly: true,
      maxAge: 24 * 60 * 60 * 1000
    })

    res.json({
      success: true,
      message: 'Login successful',
      user: { id: 1, email: email }
    })
  } else {
    res.status(401).json({
      success: false,
      message: 'Invalid credentials'
    })
  }
})
```

2. Sending Cookie with Redirect

javascript

```
app.post('/login', (req, res) => {
  // Set cookie then redirect
  res.cookie('user', 'John', { httpOnly: true })
```

```
res.redirect('/dashboard') // Cookie is sent with redirect
})
```

3. Sending Cookie with File Download

javascript

```
app.get('/download', (req, res) => {
  // Track download in cookie
  res.cookie('lastDownload', new Date().toISOString(), {
    maxAge: 30 * 24 * 60 * 60 * 1000
  })

  res.download('file.pdf')
})
```

Cookie Options Reference (When Sending)

javascript

```
res.cookie('name', 'value', {
  // EXPIRATION
  maxAge: 900000,           // Milliseconds until expiry
  expires: new Date(),      // Specific date object

  // SECURITY
  httpOnly: true,           // Can't be read by JS (prevents XSS)
  secure: true,             // HTTPS only (set false for localhost)
  sameSite: 'strict',      // 'strict', 'lax', or 'none'
  signed: true,             // Sign cookie to prevent tampering

  // SCOPE
  domain: '.example.com',   // Subdomains can access
  path: '/admin',          // Only sent to /admin routes

  // OTHER
  encode: String,           // Custom encoding function
  priority: 'high'          // 'low', 'medium', 'high' (modern browsers)
})
```

Common Patterns for Sending Cookies

Pattern 1: Set Multiple Cookies at Once

javascript

```
app.get('/set-all', (req, res) => {  
  res  
    .cookie('a', '1')  
    .cookie('b', '2')  
    .cookie('c', '3')  
    .send('Three cookies set')  
})
```

Pattern 2: Conditional Cookie Sending

javascript

```
app.post('/settings', (req, res) => {  
  const { rememberMe, theme } = req.body  
  
  if (rememberMe) {  
    res.cookie('remember', 'true', { maxAge: 30 * 24 * 60 * 60 * 1000 })  
  }  
  
  if (theme) {  
    res.cookie('theme', theme, { maxAge: 365 * 24 * 60 * 60 * 1000 })  
  }  
  
  res.send('Settings saved')  
})
```

Pattern 3: Cookie Based on Environment

javascript

```
app.get('/secure-data', (req, res) => {  
  const isProduction = process.env.NODE_ENV === 'production'  
  
  res.cookie('sessionId', generateSessionId(), {
```

```
    httpOnly: true,  
    secure: isProduction,    // Only HTTPS in production  
    sameSite: 'strict'  
  })  
  
  res.send('Secure cookie sent')  
})
```

Testing Cookie Sending (cURL Commands)

bash

```
# See cookie headers in response  
curl -I http://localhost:3000/send-simple-cookie  
  
# Send request and save cookie  
curl -c cookies.txt http://localhost:3000/login  
  
# Send request with saved cookie  
curl -b cookies.txt http://localhost:3000/dashboard
```

Debugging Tips

javascript

```
// Log when sending cookies  
app.use((req, res, next) => {  
  const originalCookie = res.cookie  
  
  res.cookie = function(name, value, options) {  
    console.log(`🍪 Sending cookie: ${name}=${value}`, options)  
    return originalCookie.call(this, name, value, options)  
  }  
  
  next()  
})
```

Summary

Sending cookies in Express:

1. **Use** `res.cookie(name, value, options)` to send cookies
2. **No parser needed** to send cookies (only to read incoming)
3. **Always use** `httpOnly: true` for authentication cookies
4. **Set appropriate expiration** with `maxAge` OR `expires`
5. **Use signed cookies** when security matters
6. **Clear cookies** with `res.clearCookie(name)`

Remember: Cookies are sent in the response headers before the body. Once you send the response, you can't add more cookies!

cookie parser

Here's a **complete guide to** `cookie-parser` - what it is, why you need it, and how to use it in Express.js.

What is Cookie-Parser?

Simple definition: `cookie-parser` is an Express middleware that automatically reads cookies from incoming requests and converts them into a usable JavaScript object.

Why Do You Need Cookie-Parser?

Without Cookie-Parser (Manual Parsing)

javascript

```
app.get('/dashboard', (req, res) => {  
  // Cookies come as a raw string: "user=John; theme=dark; Lang=en"  
  const rawCookie = req.headers.cookie  
  
  // You have to parse it manually (painful!)  
  const cookies = {}  
  if (rawCookie) {  
    rawCookie.split(';').forEach(pair => {  
      const [key, value] = pair.trim().split('=')  
      cookies[key] = decodeURIComponent(value)  
    })  
  }
```

```
}

    console.log(cookies.user) // "John"
    // This is tedious for every route!
  })
```

With Cookie-Parser (Automatic)

javascript

```
app.use(cookieParser())

app.get('/dashboard', (req, res) => {
  // Already parsed into an object!
  console.log(req.cookies.user)    // "John"
  console.log(req.cookies.theme)   // "dark"
  console.log(req.cookies.lang)    // "en"
  // Much easier!
})
```

Installation

bash

```
npm install cookie-parser
```

Basic Setup

javascript

```
const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

// Use cookie-parser middleware
app.use(cookieParser())

// Now all routes have access to req.cookies
app.get('/', (req, res) => {
  console.log('All cookies:', req.cookies)
```

```
    res.send('Check your console')
  })

  app.listen(3000)
```

Complete Cookie-Parser Examples

Example 1: Reading Different Types of Cookies

javascript

```
const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

app.use(cookieParser())

// First, set some cookies
app.get('/set-cookies', (req, res) => {
  res.cookie('username', 'JohnDoe')
  res.cookie('theme', 'dark-mode')
  res.cookie('language', 'en-US')
  res.cookie('visitCount', 1)
  res.cookie('preferences', JSON.stringify({ notifications: true, volume: 80 }))

  res.send('Cookies have been set!')
})

// Then, read them with cookie-parser
app.get('/read-cookies', (req, res) => {
  // cookie-parser puts all cookies in req.cookies
  const { username, theme, language, visitCount, preferences } = req.cookies

  let parsedPrefs = {}
  if (preferences) {
    try {
      parsedPrefs = JSON.parse(preferences)
    } catch (e) {}
  }

  res.json({
    success: true,
```

```

    cookies: {
      username: username || 'not set',
      theme: theme || 'not set',
      language: language || 'not set',
      visitCount: visitCount || 0,
      preferences: parsedPrefs
    },
    rawCookies: req.cookies
  })
})

app.listen(3000)

```

Example 2: Signed Cookies with Cookie-Parser

Signed cookies prevent tampering by adding a digital signature.

```

javascript

const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

// IMPORTANT: Provide a secret key for signed cookies
const SECRET_KEY = 'my_super_secret_key_12345'
app.use(cookieParser(SECRET_KEY)) // Pass secret here

// Set a signed cookie
app.get('/set-signed', (req, res) => {
  // Use signed: true option
  res.cookie('userId', '12345', { signed: true })
  res.cookie('role', 'admin', { signed: true })

  res.send('Signed cookies have been set!')
})

// Read signed cookies
app.get('/read-signed', (req, res) => {
  // Signed cookies are in req.signedCookies (not req.cookies)
  const userId = req.signedCookies.userId
  const role = req.signedCookies.role

  // Regular cookies still in req.cookies

```

```

const regularCookies = req.cookies

res.json({
  signedCookies: {
    userId: userId || 'not set',
    role: role || 'not set'
  },
  regularCookies: regularCookies,
  isTampered: !userId && req.cookies.userId // If regular has it but signed doesn't = tampered
})

// Demonstrate tamper detection
app.get('/show-tamper-example', (req, res) => {
  res.send(`
    <h2>Signed Cookie Tamper Detection</h2>
    <p>1. <a href="/set-signed">Set signed cookies</a></p>
    <p>2. <a href="/read-signed">Read signed cookies (works)</a></p>
    <p>3. Open DevTools > Application > Cookies and manually edit the 'userId' cookie</p>
    <p>4. <a href="/read-signed">Read again - cookie-parser will reject it!</a></p>
  `)
})

app.listen(3000)

```

Example 3: Complete Authentication System

```

javascript

const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

// Secret for signed cookies
const COOKIE_SECRET = 'your-secret-key-change-this'
app.use(cookieParser(COOKIE_SECRET))
app.use(express.json())
app.use(express.urlencoded({ extended: true }))

// Mock user database
const users = [
  { id: 1, username: 'alice', password: '123', role: 'user' },

```

```
{ id: 2, username: 'bob', password: '456', role: 'admin' }
]

// Login endpoint - sets signed cookie
app.post('/login', (req, res) => {
  const { username, password } = req.body

  const user = users.find(u => u.username === username && u.password === password)

  if (!user) {
    return res.status(401).json({ error: 'Invalid credentials' })
  }

  // Set signed cookie with user info
  res.cookie('auth', JSON.stringify({
    id: user.id,
    username: user.username,
    role: user.role
  })), {
    signed: true,      // Sign the cookie
    httpOnly: true,    // Can't be read by JavaScript
    maxAge: 24 * 60 * 60 * 1000, // 1 day
    sameSite: 'lax'
  })

  // Also set a regular cookie (not signed) for non-sensitive data
  res.cookie('displayName', user.username, {
    maxAge: 24 * 60 * 60 * 1000
  })

  res.json({ success: true, message: 'Logged in successfully' })
})

// Authentication middleware using cookie-parser
function requireAuth(req, res, next) {
  // Read the signed cookie
  const authCookie = req.signedCookies.auth

  if (!authCookie) {
    return res.status(401).json({ error: 'Not authenticated' })
  }

  try {
    const userData = JSON.parse(authCookie)
```

```
    req.user = userData
    next()
  } catch(e) {
    res.status(401).json({ error: 'Invalid authentication' })
  }
}
```

// Admin middleware

```
function requireAdmin(req, res, next) {
  if (req.user && req.user.role === 'admin') {
    next()
  } else {
    res.status(403).json({ error: 'Admin access required' })
  }
}
```

// Protected route

```
app.get('/dashboard', requireAuth, (req, res) => {
  // req.user is available from middleware
  res.json({
    message: `Welcome ${req.user.username}!`,
    user: req.user,
    displayName: req.cookies.displayName, // Regular cookie
    allCookies: req.cookies,
    signedCookies: req.signedCookies
  })
})
```

// Admin only route

```
app.get('/admin', requireAuth, requireAdmin, (req, res) => {
  res.json({
    message: 'Admin dashboard',
    adminUser: req.user,
    systemInfo: 'Sensitive data here'
  })
})
```

// Logout - clear cookies

```
app.get('/logout', (req, res) => {
  res.clearCookie('auth')
  res.clearCookie('displayName')
  res.json({ success: true, message: 'Logged out' })
})
```

```
// Debug route - see all cookies
app.get('/debug-cookies', (req, res) => {
  res.json({
    regularCookies: req.cookies,
    signedCookies: req.signedCookies,
    rawHeader: req.headers.cookie || 'No cookies in request'
  })
})

// HTML Login Page
app.get('/', (req, res) => {
  // Check if already logged in
  const auth = req.signedCookies.auth

  if (auth) {
    try {
      const user = JSON.parse(auth)
      return res.send(`
        <h2>Welcome back, ${user.username}!</h2>
        <p>You are already logged in.</p>
        <a href="/dashboard">Go to Dashboard</a><br/>
        <a href="/logout">Logout</a>
      `)
    } catch(e) {}
  }

  res.send(`
    <html>
    <head><title>Login</title></head>
    <body>
    <h2>Login</h2>
    <form method="POST" action="/login">
      <input type="text" name="username" placeholder="Username" required /><br/>
      <input type="password" name="password" placeholder="Password" required /><br/>
      <button type="submit">Login</button>
    </form>
    <p>Test accounts:</p>
    <ul>
      <li>alice / 123 (user)</li>
      <li>bob / 456 (admin)</li>
    </ul>
    <hr/>
    <h3>Cookie Debug Info:</h3>
    <p>Regular cookies: ${JSON.stringify(req.cookies)}</p>
  `)
```

```

    <p>Signed cookies: ${JSON.stringify(req.signedCookies)}</p>
  </body>
</html>
`)
})

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000')
  console.log('Try logging in to see cookie-parser in action!')
})

```

Cookie-Parser Methods Explained

1. cookieParser() - Basic (no secret)

javascript

```

app.use(cookieParser())

// Only populates req.cookies
// Cannot read signed cookies

```

2. cookieParser(secret) - With secret

javascript

```

app.use(cookieParser('my-secret-key'))

// Populates both:
// req.cookies - unsigned cookies
// req.signedCookies - signed cookies (automatically verified)

```

3. cookieParser(secret, options) - With options

javascript

```

app.use(cookieParser('secret', {
  decode: (value) => {
    // Custom decode function
  }
})

```

```
    return decodeURIComponent(value)
  }
}))
```

What Cookie-Parser Does Internally

javascript

```
// Without cookie-parser
app.get('/', (req, res) => {
  console.log(req.headers.cookie)
  // "username=John; theme=dark; sessionId=abc123"

  // You'd need to parse manually
})

// With cookie-parser
app.use(cookieParser())

app.get('/', (req, res) => {
  console.log(req.cookies)
  // { username: 'John', theme: 'dark', sessionId: 'abc123' }

  // Already parsed for you!
})
```

Common Patterns with Cookie-Parser

Pattern 1: Check if Cookie Exists

javascript

```
app.get('/check-login', (req, res) => {
  if (req.cookies.sessionId) {
    res.send('You are logged in')
  } else {
    res.send('Please login')
  }
})
```

Pattern 2: Get Cookie with Default Value

javascript

```
app.get('/preferences', (req, res) => {  
  const theme = req.cookies.theme || 'light'  
  const language = req.cookies.language || 'en'  
  
  res.json({ theme, language })  
})
```

Pattern 3: Parse JSON Cookies

javascript

```
app.get('/user-settings', (req, res) => {  
  let settings = {}  
  
  if (req.cookies.settings) {  
    try {  
      settings = JSON.parse(req.cookies.settings)  
    } catch(e) {  
      console.error('Invalid JSON in cookie')  
    }  
  }  
  
  res.json(settings)  
})
```

Pattern 4: Protect Routes with Cookie Check

javascript

```
const authMiddleware = (req, res, next) => {  
  const token = req.signedCookies.authToken  
  
  if (!token) {  
    return res.status(401).json({ error: 'Unauthorized' })  
  }  
  
  // Verify token...
```

```
    next()
  }

  app.get('/protected', authMiddleware, (req, res) => {
    res.json({ message: 'You have access!' })
  })
```

Cookie-Parser vs No Cookie-Parser

Without cookie-parser

```
req.headers.cookie = "name=John; age=25"
```

String - needs manual parsing

No signed cookie support

More code in every route

With cookie-parser

```
req.cookies = { name: 'John', age: '25' }
```

Object - ready to use

Automatic signed cookie verification

One-time setup

Error Handling with Signed Cookies

javascript

```
app.use(cookieParser('secret'))

app.get('/check-signed', (req, res) => {
  const signed = req.signedCookies.userId
  const unsigned = req.cookies.userId

  if (!signed && unsigned) {
    // Cookie was tampered with!
    console.warn('Tampered cookie detected!')
    return res.status(400).json({
      error: 'Invalid cookie detected',
      message: 'Someone tried to tamper with your cookies'
    })
  }

  res.json({ userId: signed })
})
```

Testing Cookie-Parser (cURL Commands)

```
bash
```

```
# Set cookies via curl
```

```
curl -c cookies.txt http://localhost:3000/set-cookies
```

```
# Read cookies with curl (cookie-parser parses automatically)
```

```
curl -b cookies.txt http://localhost:3000/read-cookies
```

```
# Set cookie in request header
```

```
curl -H "Cookie: username=John; theme=dark" http://localhost:3000/read-cookies
```

Common Issues and Solutions

Issue 1: req.cookies is undefined

```
javascript
```

```
// ❌ Wrong - middleware after route
```

```
app.get('/', (req, res) => {})
```

```
app.use(cookieParser())
```

```
// ✅ Correct - middleware before routes
```

```
app.use(cookieParser())
```

```
app.get('/', (req, res) => {})
```

Issue 2: Signed cookies not working

```
javascript
```

```
// ❌ Wrong - no secret provided to cookie-parser
```

```
app.use(cookieParser())
```

```
res.cookie('name', 'value', { signed: true })
```

```
// ✅ Correct - provide secret
```

```
app.use(cookieParser('my-secret'))
```

```
res.cookie('name', 'value', { signed: true })
```

Issue 3: Cookies with special characters

```
javascript

// cookie-parser automatically decodes
res.cookie('name', 'Hello World!') // Encoded automatically

app.get('/', (req, res) => {
  console.log(req.cookies.name) // "Hello World!" (decoded)
})
```

Summary

Cookie-parser does three things:

1. **Parses** `req.headers.cookie` string into `req.cookies` object
2. **Verifies** signed cookies and puts them in `req.signedCookies`
3. **Handles** decoding automatically

When to use cookie-parser:

-  Always use it if your app uses cookies
-  Use with secret for signed cookies (authentication)
-  Makes cookie handling much easier

When you might not need it:

-  Your app has zero cookies
-  You only set cookies but never read them

Bottom line: Cookie-parser is essential for any Express app that reads cookies. It saves you from writing manual parsing logic and provides security features for signed cookies.

signed parser

You're asking about **signed cookie parsing** - specifically how `cookie-parser` handles signed cookies. Let me explain it clearly.

What is a Signed Cookie Parser?

A signed cookie parser is a middleware that:

1. **Creates** cryptographic signatures for outgoing cookies
2. **Verifies** signatures for incoming cookies
3. **Rejects** cookies that have been tampered with

How Signed Cookies Work

text

```
Server sends cookie:    userId=123 + signature(123) = "123.signature"
                        ↓
Browser stores:         "123.signature"
                        ↓
Browser sends back:     "123.signature"
                        ↓
Server verifies:        Is signature valid?
                        ↓
                        YES → Use cookie
                        NO  → Reject (tampered!)
```

Simple Example Without vs With Signed Parser

Without Signed Parser (Vulnerable)

javascript

```
const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

app.use(cookieParser()) // No secret = unsigned only

app.post('/login', (req, res) => {
  // User can change this cookie value!
  res.cookie('userId', '123', { httpOnly: true })
  res.send('Logged in')
})
```

```

app.get('/admin', (req, res) => {
  const userId = req.cookies.userId

  // PROBLEM: User can manually edit cookie to "1" or "admin"
  if (userId === '123') {
    res.send('Admin access granted')
  } else {
    res.send('Access denied')
  }
})

```

Attack: User opens DevTools → Edit cookie `userId` from "123" to "1" → Gets admin access!

With Signed Parser (Secure)

javascript

```

const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

// SECRET KEY - only server knows this
const SECRET = 'my-super-secret-key-12345'
app.use(cookieParser(SECRET)) // <<< Signed parser!

app.post('/login', (req, res) => {
  // This creates a SIGNED cookie
  res.cookie('userId', '123', {
    signed: true,      // Sign it!
    httpOnly: true
  })
  res.send('Logged in')
})

app.get('/admin', (req, res) => {
  // Read SIGNED cookie (automatically verified)
  const userId = req.signedCookies.userId

  if (userId === '123') {
    res.send('Admin access granted')
  }
})

```

```

    } else {
      // If user tampered with cookie, userId will be undefined
      res.status(403).send('Access denied - Invalid cookie')
    }
  })

app.get('/debug', (req, res) => {
  res.json({
    regularCookies: req.cookies,      // The raw cookie value (with signature)
    signedCookies: req.signedCookies // Verified value or undefined
  })
})

```

How Signature Works (Under the Hood)

javascript

// Simplified example of what cookie-parser does internally

```

function signCookie(value, secret) {
  // Create a signature using HMAC
  const crypto = require('crypto')
  const signature = crypto
    .createHmac('sha256', secret)
    .update(String(value))
    .digest('base64')
    .replace(/=+$/, '')

  // Return: "value.signature"
  return `${value}.${signature}`
}

function unsignCookie(signedValue, secret) {
  const [value, signature] = signedValue.split('.')
  const expectedSignature = crypto
    .createHmac('sha256', secret)
    .update(value)
    .digest('base64')
    .replace(/=+$/, '')

  if (signature === expectedSignature) {
    return value // Valid!
  }
}

```

```

    return false    // Tampered!
  }

  // Usage:
  const secret = 'my-secret'
  const signed = signCookie('123', secret) // "123.abc123def456"
  const isValid = unsignCookie(signed, secret) // "123"
  const tampered = unsignCookie('456.abc123def456', secret) // false

```

Complete Signed Parser Example

server.js :

```

javascript

const express = require('express')
const cookieParser = require('cookie-parser')
const app = express()

// IMPORTANT: The secret used for signing
const SECRET_KEY = process.env.COOKIE_SECRET || 'development-secret-key-change-me'

// Initialize signed cookie parser
app.use(cookieParser(SECRET_KEY))
app.use(express.json())
app.use(express.urlencoded({ extended: true }))

// Mock user database
const users = [
  { id: 1, username: 'alice', role: 'user', balance: 100 },
  { id: 2, username: 'bob', role: 'admin', balance: 5000 },
  { id: 3, username: 'carol', role: 'user', balance: 50 }
]

// Login - sends signed cookie
app.post('/login', (req, res) => {
  const { username, password } = req.body

  const user = users.find(u => u.username === username)

  if (!user || password !== 'password123') {
    return res.status(401).json({ error: 'Invalid credentials' })
  }
}

```

```
// Create signed cookie with user data
const sessionData = {
  userId: user.id,
  username: user.username,
  role: user.role,
  loginTime: Date.now()
}

res.cookie('session', JSON.stringify(sessionData), {
  signed: true,      // <<< SIGNED
  httpOnly: true,    // Can't read with JavaScript
  maxAge: 24 * 60 * 60 * 1000, // 1 day
  sameSite: 'lax'
})

res.json({
  success: true,
  message: 'Logged in successfully',
  user: { id: user.id, username: user.username, role: user.role }
})
})

// Middleware to verify signed cookie
function authenticate(req, res, next) {
  // Get the SIGNED cookie (automatically verified by cookie-parser)
  const session = req.signedCookies.session

  if (!session) {
    return res.status(401).json({
      error: 'Not authenticated',
      message: 'No valid signed cookie found'
    })
  }

  try {
    const sessionData = JSON.parse(session)
    req.user = sessionData
    next()
  } catch (e) {
    res.status(401).json({ error: 'Invalid session data' })
  }
}
```

```
// Protected route
app.get('/profile', authenticate, (req, res) => {
  const user = users.find(u => u.id === req.user.userId)

  res.json({
    message: 'Profile data',
    user: {
      id: user.id,
      username: user.username,
      role: user.role,
      balance: user.balance
    },
    session: req.user
  })
})

// Admin route
app.get('/admin', authenticate, (req, res) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required' })
  }

  res.json({
    message: 'Admin dashboard',
    users: users,
    systemInfo: 'Sensitive data'
  })
})

// Logout - clear signed cookie
app.post('/logout', (req, res) => {
  res.clearCookie('session')
  res.json({ success: true, message: 'Logged out' })
})

// DEMO: Show tampering detection
app.get('/demo/tamper', (req, res) => {
  // Set a test signed cookie
  res.cookie('test', 'original-value', { signed: true })
  res.send(`
    <html>
      <body>
        <h2>Signed Cookie Tampering Demo</h2>
        <p>1. <a href="/demo/show-cookies">View cookies</a> - See the signed cookie</p>
      </body>
    </html>
  `)
})
```

```

    <p>2. Open DevTools (F12) → Application → Cookies</p>
    <p>3. Edit the 'test' cookie value to something else</p>
    <p>4. <a href="/demo/verify-tamper">Verify cookie</a> - See detection in action</p>
  </body>
</html>
`)
})

app.get('/demo/show-cookies', (req, res) => {
  res.json({
    regularCookies: req.cookies,      // Shows signature: "original-value.signature"
    signedCookies: req.signedCookies, // Shows "original-value" (verified)
    note: "The regular cookie contains the signature. The signed cookie is the verified value."
  })
})

app.get('/demo/verify-tamper', (req, res) => {
  const regular = req.cookies.test
  const signed = req.signedCookies.test

  if (!signed && regular) {
    res.json({
      status: '⚠️ TAMPERING DETECTED!',
      message: 'The cookie was modified!',
      regularValue: regular,
      signedValue: signed,
      explanation: 'The signature no longer matches the value'
    })
  } else if (signed) {
    res.json({
      status: '✅ VALID',
      message: 'Cookie is authentic and not tampered',
      value: signed
    })
  } else {
    res.json({
      status: 'No cookie found',
      message: 'Set the cookie first at /demo/tamper'
    })
  }
})

```

```
// HTML Login Page
app.get('/', (req, res) => {
  const session = req.signedCookies.session

  if (session) {
    try {
      const data = JSON.parse(session)
      return res.send(`
        <h2>Welcome back, ${data.username}!</h2>
        <p>Role: ${data.role}</p>
        <a href="/profile">View Profile</a><br/>
        <a href="/admin">Admin (if authorized)</a><br/>
        <form method="POST" action="/logout" style="display:inline">
          <button type="submit">Logout</button>
        </form>
        <hr/>
        <h3>Signed Cookie Info:</h3>
        <p>Session exists and is verified!</p>
        <a href="/debug">Debug Cookies</a>
      `)
    } catch(e) {}
  }

  res.send(`
    <html>
      <body>
        <h2>Login - Signed Cookie Demo</h2>
        <form method="POST" action="/login">
          <input type="text" name="username" placeholder="Username" required /><br/>
          <input type="password" name="password" placeholder="Password" required /><br/>
          <button type="submit">Login</button>
        </form>
        <p>Test accounts (password: password123):</p>
        <ul>
          <li>alice (user)</li>
          <li>bob (admin)</li>
          <li>carol (user)</li>
        </ul>
        <hr/>
        <h3>Learn about signed cookies:</h3>
        <a href="/demo/tamper">Tampering Detection Demo</a><br/>
        <a href="/debug">Debug Current Cookies</a>
      </body>
    </html>
  `)
```

```
`)  
})  
  
// Debug endpoint  
app.get('/debug', (req, res) => {  
  res.json({  
    regularCookies: req.cookies,  
    signedCookies: req.signedCookies,  
    hasSignedCookies: Object.keys(req.signedCookies).length > 0,  
    rawHeader: req.headers.cookie || 'No cookies',  
    note: "Regular cookies show the signature. Signed cookies are automatically verified."  
  })  
})  
  
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000')  
  console.log('\n=== Signed Cookie Parser Demo ===')  
  console.log('1. Login with alice/password123')  
  console.log('2. Check /debug to see signed vs regular cookies')  
  console.log('3. Try tampering: Open DevTools → Application → Cookies')  
  console.log('4. Edit the session cookie and refresh /profile')  
})
```

Why Use Signed Cookies?

Feature	Unsigned Cookie	Signed Cookie
Client can read	✔ Yes	✔ Yes (but can't modify without breaking signature)
Client can modify	✔ Yes (dangerous!)	✘ No (detected)
Server verification	✘ No	✔ Yes (automatic)
Security	Low (easily forged)	High (cryptographically verified)
Use case	Non-sensitive data (theme, language)	Authentication, user ID, permissions

Signed Parser Options

javascript

```
// Option 1: Basic signed parser
app.use(cookieParser('my-secret'))

// Option 2: With custom decode function
app.use(cookieParser('my-secret', {
  decode: (value) => {
    // Custom decoding before signature verification
    return decodeURIComponent(value)
  }
}))

// Option 3: Multiple secrets (for key rotation)
const secrets = ['new-secret', 'old-secret']
app.use(cookieParser(secrets))
// Will try each secret to verify signature
```

Real-World Example: JWT vs Signed Cookies

javascript

```
// Signed cookies are great for session IDs
app.post('/login', (req, res) => {
  const sessionId = generateSessionId() // Store in database

  // Send signed cookie with just the session ID
  res.cookie('sessionId', sessionId, {
    signed: true, // Cryptographically protected
    httpOnly: true
  })
})

// When verifying, cookie-parser ensures authenticity
app.get('/profile', (req, res) => {
  const sessionId = req.signedCookies.sessionId

  if (!sessionId) {
    return res.status(401).send('Invalid session')
  }
})
```

```
// Lookup session in database (safe because cookie is verified)
const userData = db.findSession(sessionId)
res.json(userData)
})
```

Common Questions

Q: Can the client see signed cookies?

Yes, but they can't modify them without breaking the signature.

Q: What happens if someone tampers with a signed cookie?

`req.signedCookies` will NOT contain that cookie (it will be `undefined`), while `req.cookies` will show the tampered value.

Q: Do I need a database with signed cookies?

Not necessarily. You can store non-sensitive data directly in signed cookies (like user preferences). For authentication, store just a session ID.

Q: Is the secret key safe in code?

No! Use environment variables:

```
javascript

const SECRET = process.env.COOKIE_SECRET || 'fallback-only-for-dev'
app.use(cookieParser(SECRET))
```

Summary

Signed cookie parser provides:

1. **Integrity** - Can't modify without detection
2. **Authentication** - Server knows cookie came from server

3. **Tamper detection** - Automatically rejects modified cookies

4. **Easy to use** - Just add `signed: true` and use `req.signedCookies`

When to use:

-  User authentication
-  Session management
-  Storing user IDs or roles
-  Any data you don't trust the client to modify

When not to use:

-  Storing large data (max 4KB per cookie)
-  Storing sensitive data (encrypt instead)
-  Truly public data (no need for signature)

The signed parser makes your cookies **tamper-proof**, not **hidden** or **encrypted**!